ARC UI · UIARC.DEV · PROOF OF CONCEPT

Program Overview

Arc UI (uiarc.dev) Proof-of-Concept Program — three tracks

Generated 2026-11-11

A three-track evaluation of Arc UI: can an AI agent really build with it, what does a flagship result look like, and what are the security implications of an agent-first UI library?

---

1. Background

Arc UI markets itself as **"The React UI library your AI agents can build with."** It ships 243 animated components and 141 blocks, installable via the shadcn CLI (`npx shadcn@latest add @uiarc/button`) or an MCP server, and it deliberately exposes an AI-facing documentation surface — an `llms.txt` index, per-component markdown docs, a public component registry, and an MCP endpoint — so that coding agents can discover, learn, and install components autonomously.

This program evaluates that claim from three angles, each delivered as a standalone, runnable POC with its own documentation.

#	Track	Question it answers	Deliverable
1	**Flagship product demo**	What can actually be built with Arc?	`flagship-dashboard/` — "Northwind" analytics dashboard
2	**Agent-buildable showcase**	Does the "agents can build with it" pitch hold up end-to-end?	`agent-buildable/` — agent-assembled screen + surface characterization
3	**Security / MCP risk**	What are the risks when a library's primary consumer is an AI agent?	`security-threat-model/` — defensive threat model + safe local demo

---

2. Track 1 — Flagship product demo ("Northwind")

Location: `flagship-dashboard/` · **Full doc:** `01-flagship-dashboard.md`

A motion-heavy SaaS **analytics dashboard** (Vite + React 19 + TypeScript) built to prove what an AI agent could produce with Arc as its component layer. Four views sit behind a sidebar and a ⌘K command palette:

- **Overview** — KPI stat row with count-up numbers and sparklines, a gradient revenue area chart (6M / 12M / YTD), a plan-mix donut chart, a channel bar chart, and an activity feed.
- **Projects** — a Kanban board whose cards glide between columns with shared-layout spring animations and fire toasts.
- **Transactions** — summary tiles plus a sortable, filterable data table with export.
- **Settings** — a light/dark theme toggle, six live accent colors, notification switches, and a destructive action gated behind a confirmation dialog.

Engineering highlights: ~19 Arc-style components (Card, Badge, Avatar, AnimatedNumber, Sparkline, AreaChart, BarChart, DonutChart, StatCard, Kanban, DataTable, CommandPalette, Toast, ThemeToggle, Sidebar, Button, Switch, Dialog, Tabs) built on Radix UI primitives. All charts are hand-built in SVG and animated with the `motion` library — no charting dependency. Theming (light/dark + 6 accents) is pure CSS custom properties persisted to `localStorage`, and every animation has a `prefers-reduced-motion` path. The production build passes clean; six screenshots (both themes) are captured in `screenshots/`.

Verdict: Arc's token system and motion primitives make a genuinely polished, cohesive product UI achievable — this is the "wow" artifact of the program.

---

3. Track 2 — Agent-buildable showcase

Location: `agent-buildable/` · **Full doc:** `02-agent-buildable.md` / `WRITEUP.md`

This track treats Arc's AI surface as the actual product and runs the full agent loop against it, then assembles a real "Create project" settings screen **using genuine upstream Arc source pulled from the public registry** (unmodified, documented props only). `npm run build` passes clean.

What Arc's AI surface actually exposes (read-only findings):

Surface	Status / auth	What an agent gets
`llms.txt`	200, ~90 KB	Catalog + install rules; an explicit **"Rules for agents"** section; 243 components / 141 blocks (121 + 22 free)
Per-component markdown (`/components/<name>/markdown`)	200	Typed prop tables, usage snippets, a11y/motion notes, **"Notes for AI"** sections
Public registry (`/r/<name>.json`)	200, **no auth**	The **real MIT component source** — this is what closes the loop
MCP server (`/api/mcp`)	**401, OAuth-gated (Clerk)**	Advertised list/search/get/install tools, but **not enumerable anonymously**

The agent loop: discover (llms.txt) → learn the exact API (markdown) → install real source (registry) → compile. For an agent with web access this works end-to-end on the free, unauthenticated surface — materially stronger than a normal library's "here are the React docs."

Verdict — holds up, with one asterisk: the frictionless path is `llms.txt` + markdown + registry, **not** the MCP server, which is the one surface a headless agent cannot reach without signing in. Pro is also the majority of the catalog (122/243 components, 119/141 blocks), though the free tier is enough for real screens.

---

4. Track 3 — Security / MCP risk

Location: `security-threat-model/` · **Full doc:** `03-security-threat-model.md` / `THREAT-MODEL.md`

An **authorized, defensive, educational** threat model for a security-labs curriculum. Scope is analysis plus a **safe, fully-local, benign** demonstration — read-only recon on the live site, no attack on the service, no exfiltration, no destructive actions. The framing is class-level ("what risks does an agent-first library introduce?"), **not** an accusation that Arc is malicious.

The premise: when a library's primary consumer is an autonomous agent with repo-write and shell access, its docs, MCP output, and source become **untrusted inputs** to that agent.

Real recon findings: the MCP server is OAuth-gated (401); there is **no user-agent/Accept cloaking** (content byte-identical to agents vs. browsers); Arc's own source scanned clean. The structural weakness is that docs and the source-bearing registry `r/*.json` have **no signing, no content hash, and no version pinning** — trust rests entirely on "it came from uiarc.dev today."

Threat model:

ID	Threat	Severity
T2	shadcn "copy source into repo" supply chain — unpinned/unsigned/unhashed source committed unreviewed	**HIGH**
T1	Prompt injection via docs / MCP output — agent-directed imperatives executed as "setup"	**HIGH**
T4	Over-trust / confused deputy — agent lends repo-write, shell, secret access to whatever the library says	**HIGH** (amplifier)
T3	MCP tool-poisoning / rug-pull / response cloaking	MEDIUM

The safe demo (`demo/naive_agent_sim.py`) runs the same mock library surface past two agents: a **naive** agent that obeys an injected instruction and commits red-flagged mock source, and a **defended** agent that treats library output as untrusted data and refuses. Mock-only, benign marker payload, nothing executed or exfiltrated. Mitigations are provided for both library authors (sign/provenance docs + registry, no instructions in docs, consistent responses) and agent users (treat library output as untrusted, human-review installed source, pin + hash, sandbox the agent).

---

5. Cross-cutting conclusions

- **The agent-first pitch is real on the free path.** An agent with web access can go from zero to compiling Arc UI code using only `llms.txt` + markdown + the public registry. This is Arc's genuine

differentiator.

- **The MCP server is the weakest link of the pitch, not the strongest.** It is OAuth-gated and unreachable by a headless agent, yet it is foregrounded in Arc's marketing. The unauthenticated registry is what actually delivers the value.
- **The same property that makes Arc agent-friendly makes it a security surface.** Agent-readable docs + "copy source into my repo" installs + an agent with write/shell access is exactly the shape of supply-chain and prompt-injection risk. Arc itself appears benign and well-run; the risk is structural to the category, and the mitigations apply to any agent-first library.
- **Net:** Arc UI is a credible, polished, genuinely agent-buildable library. Adopt it with the standard agent-safety hygiene any "install source into my repo" workflow demands — human review of installed source, pinning/hashing, and treating all library-supplied text as untrusted data.

---

6. How to navigate this program

```
uiarc-poc/
├─ README.md                # program overview + per-track status table
├─ docs/                    # this overview + each track's writeup, as .md and .pdf
├─ flagship-dashboard/      # Track 1 — Northwind dashboard
├─ agent-buildable/         # Track 2 — agent-build showcase
└─ security-threat-model/   # Track 3 — defensive threat model + safe local demo
```

Each track's folder has its own `README.md` and runnable code. This overview and each track's writeup are also provided as PDFs alongside this file.